SolarCast DE – Project Documentation

As of 8 October 2026 · Nauman Maqsood

SolarCast DE forecasts Germany's solar power generation one day ahead from weather forecasts, with 61% lower error than a same-as-yesterday baseline on a held-out year.

1. Overview

SolarCast DE predicts the hourly solar feed-in of all of Germany for the next three days, in GW, with a P10–P90 uncertainty band. It turns that forecast into practical advice (the sunniest and cheapest hours for flexible loads) and exposes it through a dashboard and an AI assistant.

It is an independent portfolio project built around the themes of SMA Solar Technology's AI roles in Kassel: renewable energy, weather-driven forecasting and agent-style AI applications. It is not an SMA product and uses no SMA data.

Item	Detail
Problem	Solar output swings with the weather; grid operators, traders and energy managers need to know tomorrow's generation today
Output	Hourly forecast (P10, P50, P90) for about 3 days, daily energy totals, best 3-hour windows
Scope	National total for Germany, hourly resolution
Language and libraries	Python 3.12, pandas, NumPy, scikit-learn, requests, joblib, anthropic, pytest
Data	Public and free, no API keys: energy-charts.info and Open-Meteo
Location on disk	`C:\Users\HP\Desktop\sma_solarcast`
Status	Forecast, dashboard and tests work; the AI assistant is written but not yet run with a live API key

2. How it works

The project is a five-step pipeline; each step is one Python module in the `solarcast` package.

1. **Collect data** (`data.py`): download measured solar generation, installed capacity and day-ahead prices, plus archived weather forecasts for ten locations. Everything is cached in `data/history.csv.gz`.
2. **Build features** (`features.py`): turn the raw weather into 25 model inputs such as mean irradiance, cloud cover and sun elevation.
3. **Train and evaluate** (`model.py`): fit three gradient-boosted quantile models (P10, P50, P90), score them on a held-out year, then refit on all data and save to `models/`.
4. **Forecast** (`forecast.py`): fetch the live weather forecast, predict the next days, add prices and measured values, and find the best 3-hour windows. Saved to `output/forecast.json`.
5. **Present** the result two ways: `dashboard.py` writes one self-contained HTML file, `output/dashboard.html`, with interactive charts and a table view; `agent.py` is an AI assistant in which

Claude answers questions by calling tools that read the saved forecast.

AI assistant tools

The assistant never invents numbers; every figure in an answer comes from one of these tools.

Tool	Inputs	Returns
`get_outlook`	none	Daily energy, peak, cloud cover and best windows for every forecast day
`get_hourly_forecast`	`date` (YYYY-MM-DD)	Hour-by-hour P10/P50/P90, measured value, price, cloud cover
`find_best_window`	`date`, `duration_hours` (1–12), `objective` (cheapest or sunniest), `earliest_hour`, `latest_hour`	Start and end of the best window with its mean solar output and price
`get_model_accuracy`	none	Held-out error metrics and baseline comparison

The assistant uses the Anthropic Python SDK's tool runner with the model `claude-opus-5-5`.

3. Data sources

All data comes from two free public APIs that need no key or registration. Both are licensed CC BY 4.0.

Data	Provider	Endpoint	Native resolution
Measured solar generation, Germany	energy-charts.info (Fraunhofer ISE)	`api.energy-charts.info/public_power?country=de`	15 minutes
Installed solar capacity (AC)	energy-charts.info	`api.energy-charts.info/installed_power?country=de&time_step=monthly`	Monthly
Day-ahead price, DE-LU zone	energy-charts.info	`api.energy-charts.info/price?bzn=DE-LU`	Hourly, 15 minutes since Oct 2025
Archived weather forecasts	Open-Meteo	`historical-forecast-api.open-meteo.com/v1/forecast`	Hourly
Live weather forecast	Open-Meteo	`api.open-meteo.com/v1/forecast`	Hourly

Weather locations

Weather is sampled at ten points: Kassel plus the regions holding most of Germany's PV capacity.

Location	Latitude	Longitude
Kassel	51.31	9.48
Munich	48.14	11.58
Nuremberg	49.45	11.08
Stuttgart	48.78	9.18
Freiburg	47.99	7.85

Location	Latitude	Longitude
Cologne	50.94	6.96
Hannover	52.37	9.73
Leipzig	51.34	12.37
Cottbus	51.76	14.33
Rostock	54.09	12.14

Time handling

- Every series is stored on an hourly UTC index; the row labelled `h` describes the interval from `h` to `h` + 1 hour.
- 15-minute generation and price values are averaged to hourly means.
- Open-Meteo labels radiation with the end of its averaging hour, so radiation columns are shifted back one hour to line up with generation.
- Local German time (Europe/Berlin) is used only for display and for daily summaries.
- History covers 2023-01-01 to 2026-09-30: 32,847 usable hours.

4. Data dictionary

The project has four data artefacts: the cached training history, the feature matrix, the saved forecast and the model files.

Training history: `data/history.csv.gz`

One row per hour, 73 columns plus the index. Loaded as a pandas DataFrame.

Column	Data type	Unit	Meaning
index	datetime64, timezone UTC	–	Start of the hour
`shortwave_radiation__<location>` (10 columns)	float64	W/m²	Global horizontal irradiance (GHI), hourly mean
`direct_normal_irradiance__<location>` (10)	float64	W/m²	Direct normal irradiance (DNI), hourly mean
`diffuse_radiation__<location>` (10)	float64	W/m²	Diffuse horizontal irradiance (DHI), hourly mean
`cloud_cover__<location>` (10)	float64	%	Total cloud cover, 0–100
`temperature_2m__<location>` (10)	float64	°C	Air temperature at 2 m
`wind_speed_10m__<location>` (10)	float64	km/h	Wind speed at 10 m
`snow_depth__<location>` (10)	float64	m	Snow depth on the ground
`solar_mw`	float64	MW	Measured German solar net generation, hourly mean

Column	Data type	Unit	Meaning
`price_eur_mwh`	float64	EUR/MWh	Day-ahead price, DE-LU zone (can be negative)
`capacity_gw`	float64	GW	Installed solar AC capacity valid in that month

Model inputs (25 features)

Built by `build_features()` from the weather columns only, so the same code serves training and live forecasting.

Feature	Data type	Meaning
`<variable>_mean` (7 features)	float64	Mean of each weather variable across the ten locations
`shortwave_radiation_std`, `cloud_cover_std`	float64	Spread across locations: how uneven the weather is
`ghi_<location>` (10 features)	float64	Irradiance per location, so the model learns where capacity sits
`snow_share`	float64, 0–1	Share of locations with more than 2 cm of snow
`sun_elevation`	float64, degrees	Sun height above the horizon at 51.0°N, 10.0°E, mid-hour
`sun_up`	float64, 0–1	Sine of the elevation, zero when the sun is down
`doy_sin`, `doy_cos`	float64	Day of year encoded as a cycle
`hour_utc`	int32, 0–23	Hour of day in UTC

Target

Name	Data type	Formula
Capacity factor	float64, 0–1	`solar_mw / (capacity_gw * 1000)`

Saved forecast: `output/forecast.json`

Field	Data type	Meaning
`generated_at`	string	Local time the forecast was made
`now_utc`	string, ISO 8601	Current hour in UTC
`capacity_gw`	number	Installed capacity used to scale the forecast
`timezone`	string	`Europe/Berlin`
`time_utc`	array of ISO 8601 strings	One entry per hour, about 119 hours (2 days back, 3 ahead)

Field	Data type	Meaning
`p10`, `p50`, `p90`	arrays of numbers	Forecast solar generation in GW
`actual`	array of numbers or null	Measured generation in GW where published
`price`	array of numbers or null	Day-ahead price in EUR/MWh where published
`cloud`	array of numbers	Mean cloud cover in %
`days`	array of objects	Per day: `date`, `energy_gwh` (p10/p50/p90), `peak_gw`, `peak_time`, `mean_cloud_cover_pct`, `prices_published`, `sunniest_3h`, `cheapest_3h`

Model files: `models/`

File	Format	Content
`quantile_models.joblib`	joblib (Python dict)	Three fitted models keyed 0.1, 0.5, 0.9, plus the feature name list
`metrics.json`	JSON	Train and test periods, error metrics, baselines, monthly error
`test_predictions.csv.gz`	CSV	Held-out P10/P50/P90 predictions and measured values in GW

5. Features and model

The model predicts the capacity factor, not GW, so it stays valid while Germany's PV fleet keeps growing. The forecast in GW is the predicted capacity factor times the installed capacity.

Setting	Value
Algorithm	`HistGradientBoostingRegressor` from scikit-learn, quantile loss
Models	Three: quantiles 0.1, 0.5 and 0.9 (P10, P50, P90); P50 is the point forecast
Hyperparameters	400 iterations max, learning rate 0.05, 31 leaves, minimum 40 samples per leaf, L2 regularisation 1.0, early stopping on
Training rows	Daylight hours only; night is set to zero by rule
Night rule	Sun elevation below −6° and zero mean irradiance
Post-processing	Clip to zero or above, sort the three quantiles so they never cross
Training period	2023-01-01 to 2025-09-30 (24,091 hours)
Held-out test period	2025-10-01 to 2026-09-30 (8,756 hours)
Final model	Refit on the full history after evaluation

Design decisions

- **Time-based split.** The test year comes strictly after the training period, which mirrors real use and avoids leaking the future into training.
- **Forecast inputs, not observations.** Training uses archived weather forecasts, so the model learns to cope with forecast errors.
- **Daylight-only training.** Half of all hours are exactly zero. Including them made the P10 model collapse to zero, so night is handled by a rule instead.
- **Two baselines.** Persistence (same hour yesterday) and a linear regression on mean irradiance show how much the model really adds.

Evaluation metrics

Metric	Meaning
MAE (GW)	Mean absolute error of the P50 forecast
RMSE (GW)	Root mean squared error; punishes large misses more
nMAE (%)	MAE as a share of installed capacity
Bias (GW)	Mean of forecast minus measured; positive means over-forecasting
Skill vs persistence (%)	Reduction in MAE compared with persistence
P10–P90 coverage (%)	Share of daylight hours where the measured value fell inside the band; ideal is 80%

6. Results and limitations

On the held-out year the model's mean absolute error is 0.76 GW, which is 0.67% of the 112 GW mean installed capacity and 61% lower than persistence.

Method	MAE (GW)	RMSE (GW)	Bias (GW)
Gradient boosting (this model)	0.76	1.57	+0.30
Linear regression on mean irradiance	1.17	2.25	+0.43
Persistence (same hour yesterday)	1.96	4.18	−0.02

All hours of 2025-10-01 to 2026-09-30. Over daylight hours only, the model's MAE is 1.49 GW (1.32% of capacity). Monthly error ranges from about 0.3 GW in winter to 1.2 GW in July; the dashboard shows the full breakdown.

Known limitations

- **Over-forecasting.** The model runs high in 11 of 12 held-out months, by up to 1.2 GW on average over daylight hours. Curtailment at negative prices and reported capacity running ahead of real feed-in are plausible causes, not confirmed ones.
- **Band slightly too narrow.** The P10–P90 band contained 75% of daylight hours against a nominal 80%.

- **Optimistic weather inputs.** The Open-Meteo archive holds short-lead forecasts, so true day-ahead accuracy will be somewhat worse than the table.
- **National scope.** The target is Germany's total feed-in. Forecasting a single plant needs inverter data, which is not public.
- **Assistant not run live.** Its four tools are verified by direct calls, but the full conversation needs an Anthropic API key that was not available during the build.

7. Getting started

You need Python 3.10 or newer and an internet connection; nothing else has to be installed or registered for the forecast and dashboard.

Install

Open a terminal in `C:\Users\HP\Desktop\sma_solarcast`. A virtual environment named `.venv` already exists there with everything installed, so on this PC you only need the activate line.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

On Linux or macOS, activate with `source .venv/bin/activate` instead.

Commands

Command	What it does	Typical duration (estimate)
`python run.py train`	Downloads the history (first run only), trains, evaluates, saves the models	A few minutes on first run, about a minute after
`python run.py train --refresh`	Same, but re-downloads the history	A few minutes
`python run.py forecast`	Fetches the live weather forecast and predicts the next days	Under a minute
`python run.py dashboard`	Writes `output/dashboard.html` and opens it in the browser	Seconds
`python run.py all`	Forecast plus dashboard	Under a minute
`python run.py ask "When is solar highest tomorrow?"`	One question to the AI assistant	Needs an API key
`python run.py chat`	Interactive chat with the AI assistant	Needs an API key
`python -m pytest`	Runs the 4 unit tests	Seconds

For the assistant, create an API key at console.anthropic.com and set it before running: `set ANTHROPIC_API_KEY=your-key` in the Windows command prompt, or `$env:ANTHROPIC_API_KEY="your-key"` in PowerShell.

Project layout

Path	Purpose
`run.py`	Command line entry point
`requirements.txt`	Python dependencies
`README.md`	Short project summary
`solarcast/config.py`	Locations, date ranges, API endpoints, quantiles
`solarcast/data.py`	API clients, time alignment, cached history
`solarcast/features.py`	Feature engineering and sun-position formula
`solarcast/model.py`	Training, evaluation, quantile prediction
`solarcast/forecast.py`	Live forecast and best-window search
`solarcast/dashboard.py`, `dashboard_template.html`	Dashboard renderer and its HTML template
`solarcast/agent.py`	AI assistant and its four tools
`tests/test_core.py`	Unit tests
`data/`, `models/`, `output/`	Generated files: cached history, trained models, forecast and dashboard

Changing the setup

All tunable settings live in `solarcast/config.py`: add or remove weather locations in `LOCATIONS`, move the training window with `HISTORY_START` and `HISTORY_END`, and move the test split with `TEST_START`. Run `python run.py train --refresh` after changing locations or dates.

8. Implementation guide

To rebuild the project from scratch, or to explain how it was built, follow these steps in order; each one can be run and checked before the next.

1. **Set up the environment.** Create the folder and virtual environment and install the packages in `requirements.txt`.
2. **Write `config.py`.** Fix the locations, weather variables, date ranges and endpoints in one place first.
3. **Write the API clients in `data.py`.** Start with one day of data per endpoint and print the shapes. Add chunked downloads, retries and the one-hour radiation shift, then cache the joined history to CSV.
4. **Write `features.py`.** Implement the sun-elevation formula and check it against a known value: about 62° at solar noon in Kassel on 21 June.
5. **Write `model.py`.** Split by time, train the baselines first, then the three quantile models. Compare against the baselines before tuning anything.
6. **Write `forecast.py`.** Reuse `build_features()` on the live weather forecast, scale by installed capacity, and add the best-window search.

7. **Write the dashboard.** Embed the forecast JSON into an HTML template so the result is one file with no server.
8. **Write `agent.py`.** Wrap the forecast functions as tools, test each tool by calling it directly, then connect the Claude tool runner.
9. **Add tests.** Cover the parts with exact answers: sun position, feature completeness, window search including gaps.

Next steps

- Run the assistant end to end with an Anthropic API key.
- Add the day-ahead price and public holidays as features to capture curtailment and reduce the positive bias.
- Calibrate the quantiles (conformal prediction) to reach the 80% coverage target.
- Train on true day-ahead forecast runs from Open-Meteo's previous-runs API.
- Build a site-level variant on inverter data that feeds a battery dispatch optimiser.
- Add document retrieval (grid codes, tariff sheets) to the assistant.
- Put the code on GitHub with the README and a dashboard screenshot.

9. CV portfolio entry

This entry follows the format of the Projects section in the CV.

10.2026 – 10.2026 · SolarCast DE: AI-Based Day-Ahead Solar Power Forecasting for Germany

- Built an end-to-end machine learning pipeline that forecasts Germany's hourly solar generation from numerical weather forecasts for ten locations, using public data from Fraunhofer ISE (energy-charts.info) and Open-Meteo.
- Trained gradient-boosted quantile regression models (P10/P50/P90) on 24,000+ hours and evaluated on a held-out year: MAE 0.76 GW (0.67% of 112 GW installed capacity), 61% lower error than a persistence baseline.
- Developed a decision-support layer that finds the sunniest and cheapest time windows for flexible loads from the forecast and day-ahead electricity prices.
- Implemented an LLM agent with tool calling (Claude API) that answers natural-language questions grounded in the forecast data, and an interactive single-file HTML dashboard.
- Technologies: Python, pandas, scikit-learn, REST APIs, Anthropic Claude API (tool use), pytest, HTML/JavaScript.

Shorter version (two lines)

SolarCast DE: day-ahead solar power forecasting for Germany with gradient-boosted quantile regression on weather forecasts; 61% lower error than persistence on a held-out year (MAE 0.76 GW). Includes an LLM tool-calling assistant and an interactive dashboard. Python, scikit-learn, Claude API.

Before using it

- Leave out the LLM agent bullet, or keep the wording at "implemented", until the assistant has been run with an API key.
- Be ready to explain the positive bias and the short-lead weather caveat; interviewers at an energy company are likely to ask.
- Add a GitHub link next to the title once the code is published.